ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл холбооны технологийн сургууль



БИЕ ДААЛТ 1 ГАРЫН АВЛАГА

Үйлдлийн систем (F.CSM302)
2026-2027 оны хичээлийн жилийн намар

Бие даалтын ажлын нэр: Dining-Philosophers-Problem
Хичээл заасан багш: Г.Ганбат
Багийн дугаар: 4-2-11
Багийн гишүүд: Оюутан 1: А.Аминхишиг B242270011
Оюутан 2: Д.Гантогтох B242270139
Оюутан 3: Т.Буяндэлгэр B222270052
Оюутан 4: Б.Билэгбарьмид B241940060
Оюутан 5: Г.Ёндонжунай B231940060
Оюутан 6: З.Билгүүдэй B200950011

Улаанбаатар
2026

Агуулга

1. Хэрэглэгчийн заавар 3
1.1 Програмын зорилго 3
1.2 Дэмжих орчин 3
1.3 Суулгах ба нээх алхам 3
1.4 Оролт ба тохиргоо 4
1.5 Ажиллуулах, зогсоох, reset хийх 5
1.6 Дүрслэлийн тэмдэг, өнгө, төлөв 5
1.7 Жишээнүүд 6
1.8 Түгээмэл алдаа ба шийдэл 6
2. Operational заавар 7
2.1 Repository clone хийх 7
2.2 OS, compiler, runtime, version 8
2.3 Dependency суулгах 8
2.4 Build ба Run команд 8
2.5 Үндсэн тестийн өгөгдөл 9
2.6 Нэмэлт boundary ба error тест 9
2.7 Хүлээгдсэн үр дүн 11
2.8 Known issues 11

1. Хэрэглэгчийн заавар

1.1 Програмын зорилго

Dining-Philosophers бол олон thread нэг нөөцийг (сэрээ) хуваалцахад deadlock-оос хэрхэн сэргийлдгийг харуулдаг синхрончлолын бодлого. N философич дугуй ширээний эргэн суух бөгөөд хөрш хоёрын дунд нэг сэрээ байна. Философич хоол идэхийн тулд зүүн, баруун хоёр сэрээг хоёуланг нь авах ёстой. Програм нь философич бүрийг thread болгон ажиллуулж (C хэл, mutex ба condition variable), төлөв бүрийн өөрчлөлтийг вэб UI дээр бодит цагт харуулна. Хэрэглэгч философичдын тоо болон бодох, хооллох хугацааг өөрчилж, ижил алгоритм өөр нөхцөлд хэрхэн ажиллахыг ажиглаж болно.

Философич гурван төлөвтэй: **THINKING** (бодож байна) → **HUNGRY** (өлссөн, сэрээ хүлээж байна) → **EATING** (хоёр сэрээг авч хооллож байна) → дахин THINKING.

1.2 Дэмжих орчин

Бүрэлдэхүүн	**Шаардлага**
Үйлдлийн систем	Linux (Ubuntu), эсвэл Windows 10/11 дээрх **WSL** (Ubuntu)
C compiler	gcc, pthread сантай
Python	Python 3.7 ба түүнээс дээш. Гадны сан хэрэггүй
Хөтөч (browser)	Chrome, Edge, Firefox
Бусад	git, GNU coreutils-ийн stdbuf (Ubuntu дээр анхнаасаа бий)

Анхаар: програм Windows-ийн энгийн командын мөр (cmd/PowerShell) дээр шууд ажиллахгүй (pthread, stdbuf байхгүй), WSL эсвэл Linux хэрэгтэй. macOS дээр stdbuf анхнаасаа байхгүй тул баталгаагүй.

1.3 Суулгах ба нээх алхам

Дэлгэрэнгүй суулгалтыг 2.3-т бичсэн. Бэлэн орчинтой бол дараах 4 алхам хангалттай (WSL эсвэл Linux terminal дээр):

```
git clone https://github.com/Amina7088/Dining-Philosophers-Problem.git

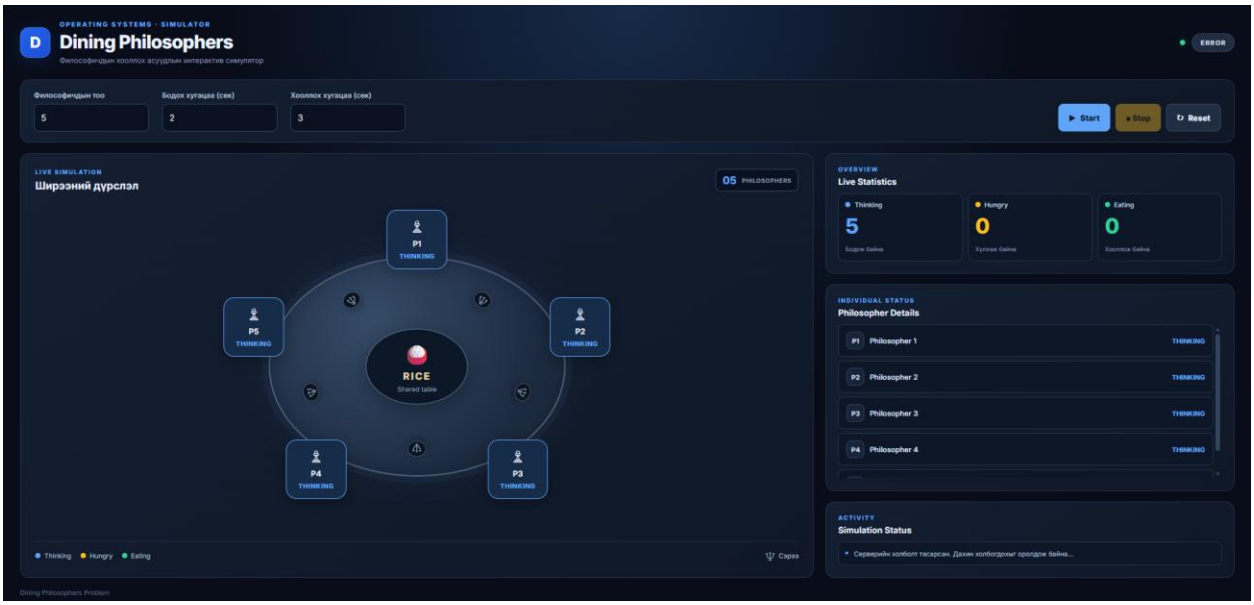
cd Dining-Philosophers-Problem

gcc -Wall -Wextra -pthread -o philosophers src/philosophers.c

python3 src/bridge/server.py --binary ./philosophers --pass-args
```

Terminal дээр Server: http://localhost:8000 гэж гарвал сервер бэлэн. Дараа нь хөтөч дээр **http://localhost:8000** хаягийг нээнэ. Windows дээр WSL-ийн сервер Windows-ийн хөтчөөс ингэж шууд нээгддэг. **index.html файлыг хоёр дарж (file://) нээж болохгүй**, учир нь UI серверээр дамжуулан C програмтай холбогддог

1.4 Оролт ба тохиргоо



Зураг 1. Програмын үндсэн дэлгэц (READY төлөв, N=5). Дээд талд тохиргоо ба товчнууд, голд дугуй ширээ, доор тоолуур, тэмдэглэгээ, төлөвийн мөр.

Талбар	**Утга**	**Анхдагч (default утга)**	**Тайлбар**
Философичдын тоо (N)	2–10, бүхэл тоо	5	Ширээн дээрх философич ба сэрээний тоо
Бодох хугацаа (секунд)	1–10	2	THINKING төлөвт байх хугацаа
Хооллох хугацаа (секунд)	1–10	3	EATING төлөвт байх хугацаа

Буруу утга (жишээ нь N=11, хугацаа 0) оруулбал UI доод мөрөнд алдааны текст гарч, програм эхлэхгүй. Тохиргооны аль нэг талбарыг өөрчлөхөд ажиллаж буй симуляци зогсож, философичид эхний төлөвт (бүгд THINKING) буцна.

1.5 Ажиллуулах, зогсоох, reset хийх

Товч	Юу хийх вэ
▶ **Start**	Тохиргоог шалгаж, С програмыг тухайн N ба хугацаагаар шинээр эхлүүлнэ. Төлөв RUNNING болно. Ажиллаж байхад Start идэвхгүй
■ **Stop**	Ажиллаж буй С процессыг зогсооно. Төлөв READY болж "Backend зогссон." гэж харагдана.
↻ **Reset**	Ажиллалтыг зогсоож, философичдыг эхний төлөвт (бүгд THINKING) буцаана. Дараа нь тохиргоог солиод дахин Start дарж болно

1.6 Дүрслэлийн тэмдэг, өнгө, төлөв

Өнгө	Тэмдэг	Утга
	THINKING, цэнхэр	Философич бодож байна, сэрээ хэрэггүй
	HUNGRY, шар	Өлссөн, хоёр сэрээ сул болохыг хүлээж байна
	EATING, ногоон	Хоёр сэрээг авч хооллож байна
	Сэрээ, саарал болон ногоон	Ашиглаагүй үед сэрээ саарал өнгөтэй байна.Харин сэрээ ашиглагдаж байвал ногоон өнгөтэй болно (хажуугийн философич хооллож байна гэсэн утга агуулна.)

Дэлгэцийн хэсэг	Утга
Тоолуур (Thinking / Hungry / Eating)	Тухайн мөчид төлөв бүрт хэдэн философич байгааг харуулна. Гурвын нийлбэр үргэлж N-тэй тэнцүү
Төлөвийн тэмдэг (badge)	**READY**: эхлүүлэхэд бэлэн / зогссон. **RUNNING**: ажиллаж байна. **ERROR**: буруу оролт эсвэл сервертэй холбогдож чадсангүй
Доод мөр (status)	Одоогийн байдал ба алдааны мессеж (жишээ нь "Backend ажиллаж байна (N=7).")

1.7 Жишээнүүд

Жишээ 1. Үндсэн тохиргоо (N=5, бодох 2, хооллох 3). Талбаруудыг өөрчлөхгүйгээр **Start** дарна. Эхлээд 5 философич бүгд цэнхэр. Ойролцоогоор 2 секундын дараа философичид шар болж, хоёр сэрээ сул байгаа нь ногоон болно. **Хүлээгдэх үр дүн:** нэгэн зэрэг хамгийн ихдээ **2** философич ногоон; хөрш хоёр философич хэзээ ч зэрэг ногоон болохгүй; хэн ч гацахгүй, философичид ээлжлэн иднэ.

Жишээ 2. Цөөн философич, хурдан (N=3, бодох 1, хооллох 1). Талбаруудыг тохируулаад **Start** дарна. **Хүлээгдэх үр дүн:** 3 философичийн хоёр нь үргэлж хөрш тул нэгэн зэрэг хамгийн ихдээ **1** философич ногоон. Төлөв хурдан солигдоно.

Жишээ 3. Олон философич (N=10, бодох 1, хооллох 2). Хүлээгдэх үр дүн: нэгэн зэрэг хамгийн ихдээ **5** философич ногоон. Ерөнхий дүрэм: нэгэн зэрэг хооллох философичийн дээд тоо нь N-ийн хагасын бүхэл хэсэг (N=5 үед 2, N=7 үед 3).

Жишээ 4. Буруу оролт. N=11 гэж бичээд Start дарвал програм эхлэхгүй, badge **ERROR** болж "Философичдын тоо 2–10 бүхэл тоо байх ёстой." гэж гарна.

1.8 Түгээмэл алдаа ба шийдэл

Шинж тэмдэг	Шалтгаан	Шийдэл
Start дарахад "Серверт хүрч чадсангүй..."	Сервер ажиллаагүй, эсвэл хуудсыг file:// гэж нээсэн	Серверийг ажиллуулж, хөтөч дээр http://localhost:8000 нээнэ
"Серверийн холболт тасарсан..."	Ажиллаж байхад сервер зогссон	Серверийг дахин ажиллуулж, хуудсыг дахин ачаална (F5)
Start дарахад ERROR: "N-ийг өөрчлөхийн тулд серверийг --pass-args горимоор ажиллуулна уу..."	Сервер --pass-args аргументгүйгээр (эсвэл --replay горимоор) эхэлсэн	Серверийг Ctrl+C-ээр зогсоож, 1.3-ын командаар (--binary ... --pass-args) дахин ажиллуулна
Start дарсны дараа шууд “Backend зогссон.” гэж гарч, философичид хөдөлдөггүй	C програм олдсонгүй (build хийгээгүй, эсвэл өөр хавтаснаас ажиллуулсан). Серверийн terminal дээр "No such file or directory" гарна	Repository-ийн үндсэн хавтсанд gcc командыг ажиллуулж, серверийг ижил хавтаснаас эхлүүлнэ

Шинж тэмдэг	Шалтгаан	Шийдэл
Серверийн terminal дээр "Address already in use"	8000 порт завгүй (өмнөх сервер ажиллаж байна)	Өмнөхийг зогсооно, эсвэл өөр порт: --port 8001 гэж ажиллуулаад http://localhost:8001 нээнэ
gcc эсвэл python3 "command not found"	Хэрэгслүүд суугаагүй	2.3-ыг дагаж суулгана

2. Operational заавар

2.1 Repository clone хийх

```
git clone https://github.com/Amina7088/Dining-Philosophers-Problem.git

cd Dining-Philosophers-Problem
```

Зам	Агуулга
src/philosophers.c	Backend (C): философич thread, mutex, condition variable, сэрээ авах/буцаах логик
src/bridge/server.py	Холболтын сервер (Python): C програмыг ажиллуулж, гаралтыг JSON event болгон UI-д дамжуулна; UI файлыг өгнө
src/frontend/	UI (index.html, script.js, style.css) ба холболтын bridge-client.js
tests/	C програмын тестүүд; results.txt файлд тестийн үр дүн
examples/sample_run.txt	C програмын бодит гаралтын жишээ (66 мөр), туршилтын өгөгдөл
docs/	Энэ Manual болон тайлан

Ажиллах бүтэц:

```
Browser (UI) <-- SSE /events -------- server.py <-- stdout ------- philosophers (C)

Browser (UI) --- POST /start, /stop -> server.py --- аргумент ---> philosophers (C)
```

Хөтөч нь C програмыг шууд унших боломжгүй тул дунд нь server.py байна. Start дарахад UI сервер рүү N ба хугацааг илгээнэ, сервер шалгаж C програмыг ./philosophers N thinkMs eatMs гэж эхлүүлнэ, програмын гаралтын мөр бүрийг JSON event болгож UI-д дамжуулна.

2.2 OS, compiler, runtime, version

Орчин	OS	gcc	Python
Орчин A	Windows 11 + WSL (Ubuntu)	15.2.0	3.14
Орчин B	Ubuntu 24.04 (Linux)	13.3.0	3.12.3

Хоёр орчинд ижил build ба ажиллуулалт хийгдсэн. Python-д гадны сан шаардлагагүй (зөвхөн стандарт сан).

2.3 Dependency суулгах

Windows: PowerShell-ийг Administrator эрхээр нээж:

```
wsl --install
```

Компьютерээ restart хийнэ. Ubuntu нээгдэж, Linux хэрэглэгчийн нэр, нууц үг асууна (нууц үг бичих үед юу ч харагдахгүй, энэ нь хэвийн). **Ubuntu / WSL дотор** хэрэгслүүдийг суулгана:

```
sudo apt update

sudo apt install -y build-essential python3 git

gcc --version

python3 --version
```

WSL дотор repository-г Linux хавтас (жишээ нь ~/) руу clone хийх нь хамгийн найдвартай. Windows-ийн диск дээр (/mnt/c/...) ажиллахад git "dubious ownership" гэсэн алдаа өгвөл алдааны мессежид гарсан git config --global --add safe.directory ... командыг ажиллуулна.

2.4 Build ба Run команд

Build (repository-ийн үндсэн хавтсанд):

```
gcc -Wall -Wextra -pthread -o philosophers src/philosophers.c
```

Анхааруулга, алдаа гарахгүй, philosophers гэсэн ажиллуулах файл үүснэ.

Run: UI-тай (үндсэн):

```
python3 src/bridge/server.py --binary ./philosophers --pass-args
```

Дараа нь хөтөч дээр http://localhost:8000. Серверийг Ctrl+C-ээр зогсооно.

Run: зөвхөн C програм (UI-гүй): ./philosophers [n] [thinkMs] [eatMs], Ctrl+C-ээр зогсооно:

```
./philosophers                # анхдагч: N=5, бодох 1000 мс, хооллох 2000 мс

./philosophers 7 300 300      # N=7, бодох 300 мс, хооллох 300 мс
```

Run: replay горим (gcc шаардлагагүй). Урьдчилж бичсэн гаралтыг UI-д тоглуулна, зөвхөн N=5:

```
python3 src/bridge/server.py --replay examples/sample_run.txt --delay 0.3
```

Серверийн сонголт	Утга
--binary ./philosophers	Ажиллуулах C програмын зам (анхдагч: ./philosophers)
--pass-args	UI-аас авсан N, хугацааг C програмд дамжуулна. Энэ аргументгүй бол зөвхөн N=5 байна
--replay FILE	C програмын оронд бичсэн гаралтыг тоглуулна (зөвхөн N=5)
--delay 0.3	Replay үед мөр хоорондын хугацаа (секунд)
--port 8000	Серверийн порт (анхдагч 8000)

2.5 Үндсэн тестийн өгөгдөл

examples/sample_run.txt нь C програмыг 20 секунд ажиллуулсан бодит гаралт (66 мөр, N=5). Мөр бүр дараах 4 хэлбэрийн нэг:

```
Philosopher 1 is THINKING

Philosopher 3 is HUNGRY

Philosopher 3 takes fork 2 and 3 (EATING)

Philosopher 3 putting fork 2 and 3 down (THINKING)
```

Үндсэн тест: replay горимоор серверийг ажиллуулж (2.4), UI дээр N=5 үед Start дарна. Философичид файлын дарааллаар төлөв солино. Мөн UI-гүйгээр ./philosophers ажиллуулж гаралтын мөрийн хэлбэр файлтай адил эсэхийг шалгаж болно. Бодит C програмын дараалал thread-ийн хуваарилалтаас хамаарч ажиллуулах бүрд өөр байдаг тул зөв байдлыг **дарааллаар биш дүрмээр** шалгана (2.7).

2.6 Нэмэлт boundary ба error тест

C програмын аргументын шалгалт (бүгд exit код 1, мессежийг stderr руу хэвлэнэ):

Команд	Хүлээгдэх мессеж
./philosophers 1	Алдаа: n (философичдын тоо) нь 2-10 хооронд байх ёстой (өгсөн утга: 1).

Команд	Хүлээгдэх мессеж
./philosophers 11	Алдаа: n (философичдын тоо) нь 2-10 хооронд байх ёстой (өгсөн утга: 11).
./philosophers abc	Алдаа: n (философичдын тоо) бүхэл тоо байх ёстой (өгсөн утга: "abc").
./philosophers 5.5	Алдаа: n (философичдын тоо) бүхэл тоо байх ёстой (өгсөн утга: "5.5").
./philosophers 5 50	Алдаа: thinkMs (бодох хугацаа) нь 100-10000 хооронд байх ёстой (өгсөн утга: 50).
./philosophers 5 1000 99999	Алдаа: eatMs (хооллох хугацаа) нь 100-10000 хооронд байх ёстой (өгсөн утга: 99999).
./philosophers 5 1000 2000 7	Хэрэглээ: ./philosophers [n] [thinkMs] [eatMs]

Exit код шалгах: команд ажиллуулсны дараа echo $? нь 1 гарна. Захын зөв утгууд: ./philosophers 2 100 100 (хамгийн бага) ба ./philosophers 10 10000 10000 (хамгийн их) хоёулаа ажиллана.

Серверийн шалгалт (сервер --pass-args горимоор ажиллаж байх үед, өөр terminal дээр):

```
curl -XPOST localhost:8000/start -d '{"n":11,"thinkMs":1000,"eatMs":1000}'

curl -XPOST localhost:8000/start -d '{"n":5,"thinkMs":50,"eatMs":1000}'

curl -XPOST localhost:8000/start -d 'xx'

curl -XPOST localhost:8000/start -d '{"n":7,"thinkMs":300,"eatMs":300}'

curl -XPOST localhost:8000/stop
```

Хүсэлт	Хариу
n=11	{"ok": false, "error": "Философичдын тоо 2–10 байх ёстой."}
thinkMs=50	{"ok": false, "error": "Хугацаа 0.1–10 секундын хооронд байх ёстой."}
Эвдэрсэн JSON (xx)	{"ok": false, "error": "JSON буруу байна."}

Хүсэлт	Хариу
n=7, 300/300	{"ok": true}, UI дээр 7 философич 0.3 секундын алхмаар хөдөлнө
/stop	{"ok": true}

Процесс цэвэрлэгдэж байгааг шалгах: Start дараад өөр terminal дээр pgrep philosophers нэг дугаар гаргана. Stop дарсны дараа дахин ажиллуулахад юу ч гарах ёсгүй. Start/Stop-ийг хэд хэдэн удаа давтаж шалгана.

Серверийн горимын шалгалт: серверийг --pass-args-гүйгээр эсвэл --replay-ээр ажиллуулаад UI дээр N=7 болгож Start дарвал "N-ийг өөрчлөхийн тулд серверийг --pass-args горимоор ажиллуулна уу..." гэсэн алдаа гарна (N=5 хэвээр ажиллана).

2.7 Хүлээгдсэн үр дүн

Бодит C програмтай (хугацааг 100–300 мс болгож хурдан ажиллуулаад) дараах дүрмүүд аль ч N дээр биелэх ёстой. N=2, 3, 7, 10 дээр 10–12 секунд ажиллуулж шалгасан үр дүн:

Дүрэм	Хүлээгдэх үр дүн
Хөрш хоёр философич зэрэг EATING байхгүй	Зөрчил 0
Нэгэн зэрэг EATING байгаа философичдын тоо	Дээд тал нь floor(N/2): N=2→1, 3→1, 7→3, 10→5
Deadlock	Гарахгүй, програм гацахгүй, философичид үргэлжлүүлэн иднэ
Философич бүр	Ажиллалтын турш ээлжлэн идэж, хэн ч байнга хоцрохгүй
Философичийн дугаар	1-ээс N хүртэл, давхардахгүй
Гаралтын формат	Дээрх 4 хэлбэрийн нэг, N өөрчлөгдсөн ч адил
Stop	C процесс зогсож, pgrep philosophers хоосон

2.8 Known issues

- **--pass-args заавал.** Үүнийг мартвал сервер зөвхөн N=5 дэмжинэ (хугацааны утга ч C програмд хүрэхгүй). Replay горим мөн зөвхөн N=5.

- **Зөвхөн Linux / WSL.** Windows-ийн cmd, PowerShell дээр шууд ажиллахгүй. macOS дээр stdbuf байхгүй тул баталгаагүй.
- **Хязгаар:** N 2–10, хугацаа 100–10000 мс (UI дээр 1–10 секунд).
- **Thread-ийн дараалал тодорхойгүй.** Ажиллуулах бүрд философичдын идэх дараалал өөр харагдана. Энэ нь алдаа биш, зөв байдлыг дүрмээр шалгана.
- **Сэрээний эзэмшил** тусдаа өгөгдөл биш, философичийн төлөвөөс гаргадаг: philosopher p нь сэрээ p-1 ба p-г хоёуланг нь эзэмшиж байж л EATING болдог.
- **Хөтөчийн tab хаахад** C процесс ажилласаар байна. Stop дарах эсвэл сервер зогсвол процесс зогсоно.
- **Порт завгүй үед** сервер Python traceback ("Address already in use") хэлбэрээр алдаа өгнө. --port-оор өөр порт сонгоно.